CSE 4345 – Big Data Analytics
Complex Engineering Problem (CEP) Assignment

EpiScope

A Big Data Surveillance, Forecasting and Response-Planning Platform
for Seasonal, Tropical and Epidemic Diseases

Group name: Modulus

Member	Student ID	Lead role
Partha Nath	0222220005101104	Data Engineer
Tahmid Jamal	0222220005101083	Analytics Engineer
Taofiq Omar Raju	0222220005101100	Visualisation Engineer

Repository: https://github.com/iparthanth/bda_pipeline (private, shared with the instructor, tag `v1.0`)
Video: https://drive.google.com/file/d/1PEtChZKgoXLoCZ7s8C676Bnu7UaYOCw3/view?usp=sharing

Department of Computer Science & Engineering, Premier University, Chittagong – Spring 2026
Instructor: Rokan Uddin Faruqui

All encounter, facility and stock data in this report are **synthetic** (teaching case); no real patient data were used.

EpiScope: A Big Data Surveillance, Forecasting and Response-Planning Platform for Bangladesh

Group Modulus: Partha Nath, Tahmid Jamal, Taofiq Omar Raju
Dept. of CSE, Premier University, Chittagong, Bangladesh

***Abstract*—EpiScope is an end-to-end prototype that turns routine health-facility encounters, climate, population and logistics data into district-level early warnings, 1–4-week forecasts of cases and hospital demand, an optimal allocation of a fixed national stock, and a seasonal preparedness calendar, served through a privacy-preserving D3.js dashboard. A seeded generator produces 28,641,734 synthetic encounters (SF = 1, 2021–2025, 64 districts, 783 facilities, 35 injected outbreaks and reporting artefacts) driven by weather; Apache Spark builds a Delta Lake medallion lakehouse with 27 quality rules, quarantine, salted hashing, idempotent incremental loads and row-level lineage. A seasonal CUSUM detector finds 88% of injected local outbreaks at 2.97 false alerts per district-year; per-district negative-binomial GLMs reduce the weighted interval score to 0.41 of a seasonal-naive baseline one week ahead; a mixed-integer programme with an explicit equity floor leaves less weighted unmet demand than a population-proportional plan. The full pipeline rebuilds from a clean clone with one command in 19.6 minutes on a 4-core, 16 GB machine.**

***Index Terms*—disease surveillance, Apache Spark, Delta Lake, outbreak detection, forecasting, optimisation, privacy, D3.js**

I. Introduction and Problem Analysis

Bangladesh carries a heavy and strongly seasonal infectious-disease burden: dengue follows the monsoon (2023 was the worst season on record), acute watery diarrhoea and cholera rise before the monsoon and after floods, influenza-like illness (ILI) peaks in the rainy season, and malaria is concentrated in the Chittagong Hill Tracts. The data needed to act early exist but are scattered across facility reporting (DHIS2-style), laboratories, the meteorological department, the census and district stores, with different formats, delays and quality. The client brief asks for a platform that, for every district, (i) detects unusual rises early, (ii) forecasts cases and hospital demand 1–4 weeks ahead, (iii) recommends how to allocate a limited national stock and rapid-response teams, and (iv) produces a preparedness calendar – explainably, robustly to late and incomplete data, on modest infrastructure, and without exposing identifiable patients.

Requirements. We translated the brief into the catalogue of 36 requirement IDs (ING, DQ, PRV, AN, VIS, ENG) and tracked every one in the repository README (ID $\rightarrow$ file $\rightarrow$ report section). All 23 Must and all 9 Should items are implemented, and so are the 4 Could items: ING-7 (Delta time travel), PRV-4 (k-anonymity), AN-10 (SEIR) and VIS-7 (mobile layout and animated playback).

Stakeholders (Table I) have conflicting needs, which drive five explicit conflicts (C1–C5, Section II). The key analytical difficulty is *interdependence*: ingestion quality determines detection, detection and forecasts feed the optimiser, and the optimiser's uncertainty must be visible in the dashboard, so an error in one stage propagates through all of them.

TABLE I
Stakeholders, needs and how EpiScope serves them

Stakeholder	Primary need / concern	EpiScope feature
MoH policymakers	national picture; defensible allocation ("why X over Y?")	optimiser with stated weights, baseline comparison, policy brief
Epidemiologists	reliable, transparent signals	two methods, trade-off curve, artefact guard, evaluation panel
District officers	early local warning; alert fatigue	ranked table, per-district forecasts, district role view
Hospital managers	bed/consumable demand with uncertainty	demand quantiles, 50/80/90% bands
Vector control	timing of larval campaigns	preparedness calendar (onset − 8 weeks)
Patients & public	no misuse of data	salted hashing, suppression, aggregates only
Privacy regulator	minimisation, small-cell suppression	PRV-1..4, audit log, role tokens

II. CEP Attributes and Conflict Resolution

Table II maps each Complex Engineering Problem attribute to concrete design decisions.

Activity attributes. *A1 (range of resources):* two weeks, three people, one 16 GB machine in Spark local mode and no paid cloud; the scaling experiment shows SF = 1 fits. *A2 (interaction):* one configuration file and one DAG tie the generator, lakehouse, analytics and serving together, and the dashboard reads only the exported bundles, so every stage is tested against the next. *A3 (innovation):* climate (lagged station rainfall and temperature), clinical encounters and stock levels are combined in one allocation model with an explicit equity floor. *A4 (consequences):* a missed outbreak or a wrong shipment costs lives, so the operating point, the equity rule and every forecast carry stated uncertainty, and privacy is enforced by design (Section IV-F). *A5 (familiarity):* course tools (Spark, Delta, D3) applied to epidemiology, where we had to learn surveillance statistics, reporting delays and interval scoring.

C1 sensitivity vs. specificity. We evaluated four detectors over a threshold grid against the ground-truth file (which the detector never reads) and chose the operating point CUSUM ≥ 17.0 with the rule "highest sensitivity with at most

TABLE II
HOW THE SOLUTION DEALS WITH EACH CEP ATTRIBUTE (P1–P7)

P1 Depth of knowledge	Distributed processing (Spark SQL, window functions, grouped pandas UDFs, AQE, broadcast joins), columnar ACID storage (Delta/Parquet, partition pruning, column pruning), surveillance statistics (EARS C1–C3, historical limits, CUSUM), probabilistic forecasting (GLM with negative-binomial dispersion, WIS), mixed-integer optimisation (HiGHS), STL decomposition, SEIR modelling, privacy (salted hashing, complementary suppression, k-anonymity), visual encoding and interaction (focus+context, linked brushing, colour-blind-safe palettes).
P2 Conflicting requirements	Five conflicts resolved with numbers, not opinion: C1 trade-off curve and operating point (§VI-A); C2 district×week aggregation with primary+complementary suppression (§IV-F); C3 efficiency–equity curve with a recommendation rule (§VI-C); C4 daily batch + 5-minute streaming path (§III-C); C5 GLM vs gradient boosting backtest (§VI-B).
P3 Depth of analysis	No single correct answer: thresholds, horizons, lag windows (rain 4–8 weeks), dispersion estimation, the equity floor α and the planning quantile were chosen by backtests and sensitivity analysis against baselines (EARS vs CUSUM, GLM vs seasonal naive vs HGB, optimiser vs population share).
P4 Unfamiliar issues	Reporting delays (0–21 days) handled by a reporting triangle and nowcast; late records update past epi-weeks (MERGE + windowed recompute); a facility that goes silent for 3 weeks and back-fills, and a large facility that starts reporting mid-stream, are both injected and handled; 2% resubmitted records and cross-facility duplicate visits are de-duplicated; Dhaka-driven skew handled by broadcast joins and AQE.
P5 Beyond standard codes	No standard maps these data to an allocation plan: we combined WHO IDSR alert/action logic, ICD-10 coding, data-protection principles (minimisation, purpose limitation, suppression) with our own formulation (valued unmet demand, equity floor, integer teams).
P6 Stakeholders	Seven stakeholder groups with different needs (Table I); role-based views (national vs district), plain-language policy brief (Appendix A), usability protocol with non-CS users (§VII).
P7 Interdependence	One code path end to end: the same silver rules feed detection, forecasting and streaming; detection sets optimiser weights and team demand; forecast quantiles set planning demand; every gold row carries the bronze batch IDs it came from, so a bad input batch can be traced to every output it touched.

three false alert events per district-year over the four diseases" – fewer than one per disease per year, a workload a district officer can verify by phone – with ties on sensitivity broken by the shorter median delay (breaking them by fewer alerts would always pick the slowest threshold on a plateau, although every tied threshold already meets the budget; Section VI-A). **C2 granularity vs. privacy.** Upazila maps would be more actionable but 1–4-case cells identify people; we publish district×epi-week counts with primary and complementary suppression, and only aggregates for districts a user is not responsible for. **C3 efficiency vs. equity.** The equity floor α guarantees each district at least α times its population share of every resource shipped; the curve in Section VI-C shows what each α costs. **C4 freshness vs. cost.** Allocation and forecasting decisions are weekly and reporting delay is days, so a daily incremental batch is the main cadence; a Structured Streaming path refreshes alerts within five minutes for emergencies at the cost of a long-running process. **C5 accuracy vs. interpretability.** Pooled over all districts and diseases, the gradient-boosting model is more accurate at every horizon (relative WIS 0.35 vs 0.41 at one week, 0.80 vs 0.94 at four), though not for every disease (for malaria the GLM is better, Section VI-B); its 80% intervals over-cover (87%), i.e. they are wider than needed, and it gives no per-district explanation. We keep the per-district GLM, whose coefficients (lagged rain, temperature, last season) officials can read and challenge and whose intervals are calibrated (81%), as the operational model, and run the boosted model as a monitored challenger whose point forecasts are shown to analysts; this trades some accuracy for trust, the explicit C5 choice.

III. ARCHITECTURE AND DESIGN DECISIONS

Fig. 1 shows the medallion lakehouse, analytics and serving layers; Table III lists the main alternatives we considered.

A. Processing and storage

All heavy work runs in PySpark 3.5 (`ING-2`); pandas appears only inside grouped UDFs on per-district series and for final aggregated exports. Every layer is a **Delta Lake** table (Parquet + transaction log) registered in a Hive catalog backed by Derby (`ING-3`): `episcope_bronze`, `_silver` and `_gold` databases. Delta gives ACID MERGE (idempotent upserts), `replaceWhere` (recompute only affected epi-weeks) and time travel (`ING-7`). Setting `paths.lake_root` to an `s3a://` URI switches storage to MinIO through the S3A connector (`ING-5`; MinIO service in `docker-compose.yml`).

B. Partitioning

Silver encounters are partitioned by `(epi_year, adm1_pcode)`: almost every query restricts the period, and allocation and division dashboards restrict the division; 6 years × 8 divisions gives 48 directories whose files stay large (tens of MB) at SF = 1. Partitioning by district would create 384 directories with many small files and skew (Dhaka alone holds a large share of the rows); by epi-week it would create thousands of tiny files. Gold marts are small and partitioned by `epi_year`; bronze is partitioned by load batch so that silver reads exactly one batch.

C. Cadence (C4) and streaming

The operational cadence is a daily incremental batch (`make drop`); decisions it supports (weekly forecasts, fortnightly shipments) do not need fresher data, and the batch is cheap. For emergencies a Structured Streaming job (`ING-6`, `pipeline/streaming/stream_alerts.py`) watches a file inbox and, in each micro-batch, reuses the *same* bronze/silver/gold code and detection, refreshing `alerts.json`; the file-to-alert latency is logged (§VI-E).

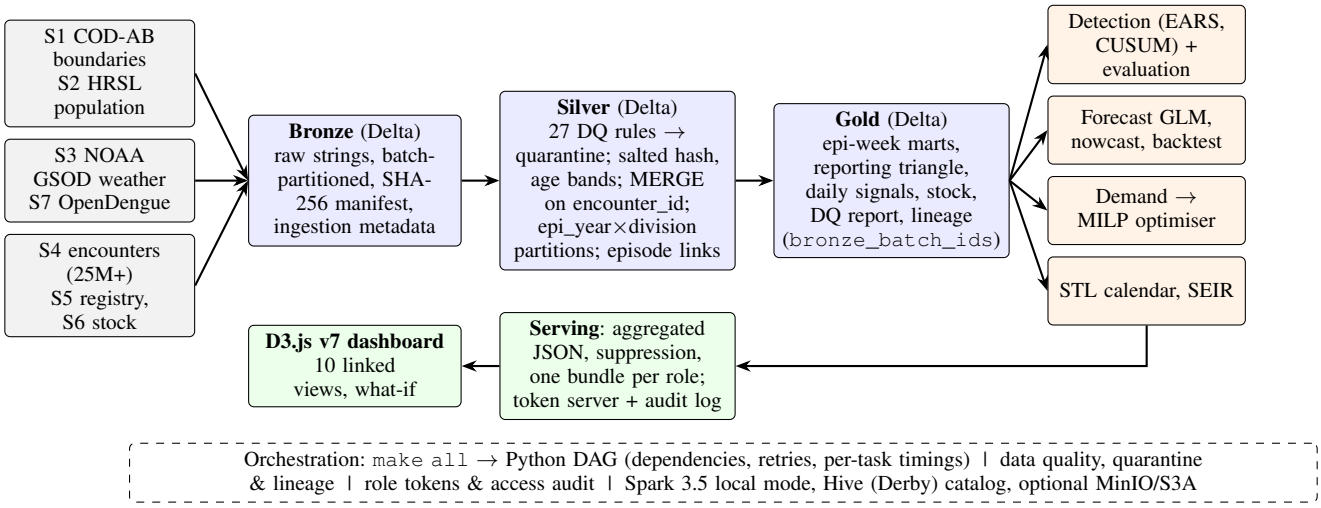


Fig. 1. EpiScope architecture (medallion lakehouse + analytics + D3.js serving layer).

TABLE III
ALTERNATIVES CONSIDERED

Decision	Chosen	Alternative (why not)
Table format	Delta Lake	plain Parquet (no MERGE/time travel); Iceberg (heavier catalog set-up)
Storage	local FS / MinIO	HDFS in Docker (more RAM on a laptop)
Orchestration	Makefile + DAG runner	Airflow/Prefect (services and memory for a 6-task DAG)
Detection	CUSUM on historical limits	Farrington flexible (needs years of clean baseline per series; slower)
Forecast	per-district NB GLM	pooled HGB (more accurate overall, opaque, over-wide intervals: challenger); ARIMA (no count noise, weather lags awkward)
Optimiser	SciPy HiGHS MILP	PuLP/CBC (extra binary); heuristics (no optimality guarantee)
Serving	stdlib server + static JSON	database-backed API (not needed for $<$10 MB of aggregates)

D. Reproducibility and orchestration

`make all` creates a virtual environment from a fully pinned `requirements.txt`, writes a git-ignored `.env` with a random salt and role tokens, and runs a small DAG runner (`orchestration/dag.py`): tasks with dependencies, one fresh JVM per stage, retries with exponential back-off and a JSON run record with per-task timings (`ENG-1`, `ENG-4`). A Dockerfile and `docker-compose.yml` give the same result in containers. Nothing hard-codes dates, file names or row counts: files are recognised by their header and the as-of date is the latest report date in silver.

IV. Data Generation, Quality, Lineage and Privacy

A. Public data (S1–S3, S7)

District (ADM2, 64) and upazila (ADM3, 507) boundaries and centroids come from OCHA COD-AB v03 (HDX), simplified to a 55 kB TopoJSON for the browser; the OpenDengue V1.3 Bangladesh series [19] checks the synthetic seasonality. Because weather drives the synthetic epidemiology, S2 and S3 are *pinned real snapshots* committed in `data/reference`, so every clone generates the same data as this report (a design decision: our build machine could not reach Open-Meteo, HDX or WorldPop, and a live download would change every result). **Weather (S3)**: raw daily NOAA GSOD files [21] of 38 Bangladesh Meteorological Department stations, stored unmodified and ingested as raw bytes; silver converts units, derives relative humidity from dew point (Magnus formula) and interpolates to district centroids (inverse-distance squared, four nearest stations within 150 km). The mirror ends on 24 Aug 2025; the remaining 13% of district-days use each district's day-of-year climatology of the same stations and are labelled `climatology_fill`. **Population (S2)**: Meta/CIESIN HRSL layers [22] (total, men, women, under-5, 15–24, 60+; about 12 GB of 30 m cells) summed on a 0.01° grid and assigned to the COD-AB districts by point in polygon (`generator/build_reference.py`); national total 164.5 M. The 5–14/25–44/45–64 split of the remaining ages uses stated fixed proportions (an assumption). Provenance (*real*, *reference* or *synthetic fallback*) is recorded per source and shown on the dashboard: boundaries real, weather real, population real. `fetch_public -refresh` downloads Open-Meteo [23] and the HDX COD-PS census table instead where reachable.

B. Synthetic encounter generator (S4–S6)

`generator/generate.py` is parameterised by a scale factor and a seed and is deterministic: each (district, year) chunk has its own `SeedSequence`, so output is byte-identical for any number of worker processes (tested). At SF = 1 it writes 28,641,734 encounter rows reported in 2021–2025 plus 197,977 rows in 14 future daily drops, a registry of 783 facilities placed on real upazilas (district hospitals, upazila health complexes, medical colleges, Dhaka-heavy private clinics) and daily JSON stock and bed-occupancy snapshots. Expected daily counts per district and disease are: dengue

driven by 4–8-week lagged rainfall and 3–6-week temperature suitability, amplified in large cities; diarrhoea with pre- and post-monsoon peaks and spikes 3–14 days after extreme rainfall; ILI with a rainy-season and a winter peak; malaria concentrated in the hill districts. Counts are drawn from a negative binomial (Poisson–gamma). We injected 35 outbreaks: 22 sharp local outbreaks of different size and duration, 3 slow-growing ones, a 2023-like dengue epidemic in five cities, post-flood cholera in Sylhet, two outbreaks that begin in the future drops, and a nationwide novel-respiratory surge spreading from Dhaka with distance. Realism defects: ≈2% resubmitted `encounter_id`s, 0.5% orphan facility keys, ≈1% invalid values, a right-skewed 0–21-day reporting delay with fewer reports on Fridays, Saturdays and holidays, a facility silent for 21 days that then back-fills, and a large private network that starts reporting mid-stream. Truth files (outbreaks, artefacts, duplicate pairs) are written to `data/ground_truth`, which only the evaluation module reads (enforced by a test).

C. Ingestion and incremental, idempotent loads

Bronze (`ING-1`) stores every row unmodified as strings plus `_source, _load_ts, _batch_id, _file_name, _file_checksum`; CSV columns are matched by name, so re-ordered or extra columns are tolerated. The SHA-256 of every file is kept in a manifest, and a file already seen is skipped. Silver MERGEs on `encounter_id` (keeping the earliest report date as `first_report_date`); gold is recomputed with `replaceWhere` from the Monday of the earliest onset week in the batch. Hence re-running a day changes nothing and late records (up to 21 days, and back-filled ones beyond) update the epi-weeks they belong to (`ING-4`); an integration test loads a drop, reloads it, force-reloads it and checks that silver and gold are identical.

D. Data quality (DQ-1, DQ-2, DQ-4)

27 rules are evaluated as Spark column predicates (16 on encounters: schema, IDs, referential integrity, dates, onset≤report, plausible delay, date range, age, sex, ICD-10 format and category, booleans, outcome, geography; 6 on the registry; 5 on stock). A row failing any rule goes to `silver.quarantine` with all its reason codes; the per-load report (rows in/out, quarantined, duplicates, pass rate per rule) is a gold table and a dashboard view. In the SF = 1 load 446,927 of 28,641,734 encounter rows (1.56%) were quarantined and 552,161 resubmissions removed. Cross-facility duplicate visits (`DQ-4`) are linked when the salted hash, disease group, sex and adjacent age band agree and onsets are at most 7 days apart; against 130,822 injected duplicate pairs (3% with a wrong sex, 30% with a mistyped age) precision was 99.998% and recall 96.9%, and 0 genuinely separate episodes were merged.

E. Lineage (DQ-3)

Every gold row carries `bronze_batch_ids`, the sorted set of bronze batches whose records produced it, and `gold.lineage_batches` maps each batch to its files, checksums and row counts; a test checks that every contributing batch exists in the manifest.

TABLE IV
K-ANONYMITY OF A MALARIA RESEARCH EXTRACT (K = 5)

Level	Records	Classes	$k_{\min}$	Suppressed	≤1%
L0	95,979	18292	1	23.6%	no
L1	95,979	8225	1	9.4%	no
L2	95,979	3214	1	3.6%	no
L3	95,979	2029	1	2.2%	no
L4	95,979	857	1	0.7%	yes
L4-released	95,274	–	5	0.7%	yes

L0 district × week × 6 age bands × sex; L1 month; L2 division; L3 3 age bands; L4 quarter.

F. Privacy by design (PRV-1..4)

Patient IDs are replaced in silver by SHA-256(salt ‖ ID) with a salt that lives only in the git-ignored `.env`; exact ages become six bands; the quarantine stores hashes too (`PRV-1`). The browser receives only aggregates: primary suppression hides every count from 1 to 4, and complementary suppression hides a second cell whenever a published district→division→national total would reveal a single hidden cell (iterated to a fixed point); no totals across weeks or diseases are published. An automated audit replays every exported hierarchy and scans every count field (`PRV-2`). Role tokens select a bundle: national users see all districts, a district user sees their district in detail and everything else only as division aggregates (`PRV-3`); each access is logged. A k-anonymity check on a record-level research extract (`PRV-4`, Table IV) climbs a generalisation ladder until $k \geq 5$ with $\leq 1\%$ suppression. For public releases we recommend differential privacy instead: adding Laplace noise with scale $1/\varepsilon$ to weekly district counts (sensitivity 1 per person-week) protects against linkage attacks that k-anonymity cannot stop, at the cost of bias in small districts; $\varepsilon \approx 1$ per released table and a privacy budget per year are reasonable starting points.

V. Analytics Methods

A. Outbreak detection (AN-1)

Detection runs daily per district and disease on counts by first report date. The *guarded* signal excludes duplicate episodes and back-filled records (records reported on the first day a normally daily facility reports again after ≥7 silent days, with delay ≥3 days); days on which more than 15% of the records come from a facility that started reporting in the last 56 days are flagged "verify reporting" rather than alerted. Weekday reporting effects are removed with national weekday factors. **EARS C1/C2/C3** [1] use Spark window functions: $z_t = (x_t - \bar{x})/s$ over a 7-day baseline (C2/C3 with a 2-day guard band; C3 sums positive C2 exceedances over 3 days), with $s \geq \max(\sqrt{\bar{x}}, 0.5)$ for sparse series. The **seasonal CUSUM** runs in a grouped pandas UDF: the expected 7-day count is the historical limit [3] (mean of the same window ±2 weeks in each earlier year) multiplied by this year's level over the last 2–7 weeks (2-week guard band), the daily count

is standardised by $\sqrt{\mu\hat{\phi}}$ with a prospective quasi-Poisson dispersion $\hat{\phi}$ (mean squared Pearson residual over the past year, winsorised), and $C_t = \max(0, C_{t-1} + z_t - 0.5)$ [4]. An alert also needs at least 5 cases in the last 7 days.

B. Forecasting and nowcasting (AN-4)

For every district and disease and each horizon $h \in \{1..4\}$ weeks, a direct log-linear Poisson model (ridge $\lambda = 0.02$) uses the last two weeks (log), the mean of weeks $t-5..t-2$, the same season last year, annual harmonics of the target week, rainfall summed and temperature averaged over weeks $t+h-8..t+h-4$ (the 4–8-week lag), and the recent maximum daily rainfall. A negative-binomial size estimated by moments gives predictive quantiles. The 256 series are trained in parallel with `groupBy(district, disease).applyInPandas`. Because the last 3–4 weeks are incomplete, the counts known at the origin are rebuilt from the reporting triangle (onset week×report week) and divided by the national completeness at that delay, estimated only from weeks 8–60 weeks before the origin. Baseline: seasonal naive (same week last year) with empirical error quantiles; comparator: gradient-boosted trees pooled over districts on $\log(1+y)$.

C. Demand conversion (AN-6)

Weekly case quantiles are sampled (1,000 Monte-Carlo draws by inverse-CDF interpolation), admissions are binomial with the district's admission rate over the last 52 weeks shrunk to the national rate (50 pseudo-cases), and converted with stated assumptions: 6 bags of IV saline per admitted dengue case and 8 per admitted diarrhoea case (+0.3 per outpatient diarrhoea case), 1.3 NS1 kits per suspected dengue case, 10 ORS sachets per diarrhoea case, 1 artemisinin course per malaria case, and length of stay 3–5 days for bed-days. The mean and 10/50/80/90% quantiles over 4 weeks are kept.

D. Allocation optimiser (AN-7)

With $\hat{y}_{d,r}$ the 80th percentile of demand, $s_{d,r}$ the latest stock, c_r unit cost, p_d population share and $w_d = 1 + [\text{alert}] + 0.5[\text{forecast growth} > 20\%]$:

$$\begin{aligned}
&\min \textstyle\sum_{d,r} w_d h_r (u_{d,r} + \beta q_{d,r}) + \sum_d w_d H_T v_d + \epsilon \sum_{d,r} c_r x_{d,r} \\
&\text{s.t. } u_{d,r} \geq \hat{y}_{d,r} - s_{d,r} - x_{d,r}, \quad q_{d,r} \geq u_{d,r} - g_{d,r}/2, \\
&\quad \textstyle\sum_{d,r} c_r x_{d,r} \leq B, \\
&\quad x_{d,r} \geq \alpha p_d \textstyle\sum_{d'} x_{d',r}, \quad \textstyle\sum_{d \in k} \sum_r m_r x_{d,r} \leq T_k, \\
&\quad v_d \geq \tau_d - z_d,\ \textstyle\sum_d z_d \leq N,\ z_d \in \mathbb{Z}_{\geq 0},\ x, u, v \geq 0.
\end{aligned}$$

Unmet units are valued in patient-equivalents h_r = share of one patient's need a unit covers × a severity weight (life-threatening 3, severe 2, moderate 1, diagnostic 0.3): IV bag $\frac{1}{6} \times 3 = 0.5$, ACT course $1 \times 2 = 2$, ORS sachet $\frac{1}{10} \times 1 = 0.1$, NS1 kit $\frac{1}{1.3} \times 0.3 = 0.23$; a missing team $H_T = 20$. These weights must be agreed with clinicians; they make sachets, kits and courses commensurable. The part of a shortfall beyond half of the district's gap $g_{d,r}$ costs $(1+\beta)$ times more ($\beta = 1$): the first half of every gap – the life-saving minimum – is filled first. This convex penalty also removes a degeneracy we observed: with purely linear valuation every taka has the same value in every district, and the solver split ties arbitrarily (a high-need district could get nothing). τ_d is the number of diseases alerting in district d (rapid-response team demand, $N = 20$ teams); m_r are unit weights and T_k a truck-capacity limit per division. α is the policy lever of conflict C3: the guaranteed fraction of each district's population share of every resource shipped. The MILP (≈640 variables) is solved with HiGHS [13] in well under a second. Baseline: the same budget split across resources by national gap and across districts by population.

E. Preparedness calendar (AN-9)

Weekly counts per division and disease are decomposed with robust STL [12] (period 52, log scale) in a grouped UDF; the seasonal component averaged by ISO week gives a multiplicative seasonal index. The season is the longest circular run of ≥3 weeks with index ≥ 1.15; campaigns start 8 weeks (dengue larval control), 6 (malaria) or 4 weeks before onset, supplies are pre-positioned 4 weeks before onset and staff surge 2 weeks before the peak, projected onto the next 52 weeks.

F. Epidemic scenarios (AN-10)

The novel-respiratory surge is located from data alone (national ILI above 130% of the same-week median of other years). A deterministic SEIR model (latent 3 days, infectious 5 days) is fitted to the weekly excess by least squares on the square-root scale (free: β, I_0, reporting fraction ρ), giving $R_0 = 2.07$, $\rho = 0.28\%$ and $R^2 = 0.997$. What-if scenarios reduce β by 20% or 40% from the detection week, two weeks earlier or later (Table X).

VI. Evaluation

All results are from the SF = 1 build (28,641,734 encounters), evaluated prospectively over 2022-01-01–2025-12-31 (the first year is the baseline burn-in).

A. Detection (AN-2, AN-3, C1)

Definitions: an outbreak (district×disease) is detected if an alert falls between 3 days before its start and 7 days after its end; days to detection count from the start; an *alert event* is a run of consecutive alert days, false if it overlaps no outbreak window. Local outbreaks (96 rows including the 64 district rows of the national surge) and the surge are reported separately. Fig. 2 and Table V give the trade-off. At the chosen operating point CUSUM ≥ 17.0 the pipeline detects 88% of local outbreaks with a median of 9 days after onset at 2.97 false alerts per district-year (all four diseases together), and flags 100% of districts in the national surge (median 93 days). **Reporting artefacts (AN-3).** In the back-fill district and window the naive signal produced 13 alert days against 0 for the guarded pipeline; around the onboarding facility, 0 vs 0 at the operating point (at lower thresholds the new-facility flag routes such days to "verify reporting" instead

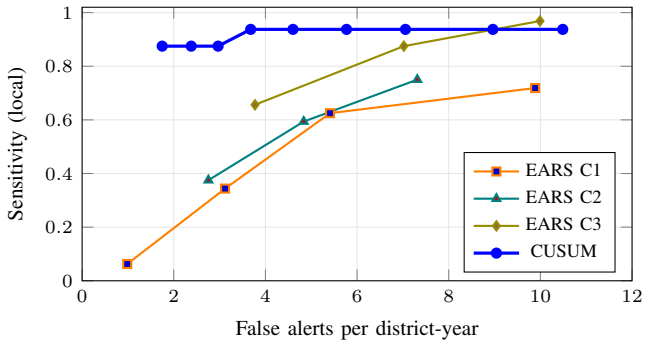


Fig. 2. Detection trade-off (guarded pipeline), one point per threshold.

of "alert"). **Interpretation.** The seasonal CUSUM dominates the EARS family everywhere on the curve: at a comparable false-alert burden (at most 4 per district-year) EARS C1 finds 34%, C2 38% and C3 66% of local outbreaks, CUSUM 94%. EARS compares a day with the previous week only, so every seasonal upswing and every post-flood spike looks anomalous; CUSUM compares with the same season in earlier years and accumulates small persistent excesses. The price is delay: 9 days at the operating point versus 1.5–5.5 days at thresholds that produce 10–20 false alerts per district-year – this is conflict C1 in numbers, and the what-if slider lets epidemiologists move along it. The national surge is detected in every district, but late relative to its injected start (93 days): the start is defined as the day seeding began, when only a few infections per million existed, so the first weeks carry no visible excess. Three design errors were found and fixed by this evaluation (AI_USAGE.md): a too-narrow level adjustment that saturated CUSUM in epidemic years, CUSUM increments computed on overlapping 7-day sums (strongly autocorrelated, which inflates false alerts), and a median-based dispersion estimate that under-estimated variance.

B. Forecasting backtest (AN-5, C5)

Rolling-origin backtest over the last 52 complete weeks (models refitted every 4 origins on data complete at the origin; nowcast inputs), scored on identical cells for all models (Table VI). Averaged over all districts and diseases, the GLM's one-week-ahead MAE is 36.3 vs 64.7 for the seasonal naive and 30.6 for pooled boosting; its WIS is 0.41 of the baseline's at one week and 0.94 at four weeks, and 80% intervals cover 81% (h = 1) and 78% (h = 4) of outcomes, close to the nominal 80% (seasonal naive: 82%). Skill decays with horizon as expected; the largest one-week gains are for ILI and dengue, whose weather-driven and multi-year patterns the naive method cannot follow, but four weeks ahead the ILI GLM is no better than the naive method. Pooled over all districts and diseases, the boosted model is more accurate at all horizons (relative WIS 0.35 at $h = 1$, 0.80 at $h = 4$), because it shares information across districts, but not for every disease: for malaria the GLM has the lower WIS at all four horizons ($h = 1$ and $h = 4$ in Table VI), and for dengue one week ahead the two are level. Its intervals also over-cover (87% at $h = 1$) and it cannot be explained district by district; we keep the GLM for decisions (C5) and log the challenger, whose accuracy advantage is the main argument for a hierarchical model in future work.

TABLE V
DETECTION OVER THE THRESHOLD GRID (* = OPERATING POINT)

Method	Thr.	Sens.	Delay	FA/DY	FA naive	NRS
EARS C1	1.5	100%	2	81.72	81.93	100%
EARS C1	2	100%	4	54.31	54.09	100%
EARS C1	2.5	100%	7	34.95	34.94	100%
EARS C1	3	97%	9	22.22	22.19	100%
EARS C1	3.5	88%	11	14.58	14.52	100%
EARS C1	4	72%	11	9.89	9.80	100%
EARS C1	5	62%	13	5.41	5.37	100%
EARS C1	6	34%	24	3.12	3.14	95%
EARS C1	8	6%	26	0.99	0.97	70%
EARS C2	1.5	100%	2	71.88	71.85	100%
EARS C2	2	100%	4	50.92	50.87	100%
EARS C2	2.5	100%	5	35.24	35.20	100%
EARS C2	3	100%	6	24.54	24.54	100%
EARS C2	3.5	100%	7	17.61	17.47	100%
EARS C2	4	94%	8	12.59	12.51	100%
EARS C2	5	75%	8	7.31	7.19	100%
EARS C2	6	59%	8	4.84	4.90	100%
EARS C2	8	38%	26	2.76	2.71	89%
EARS C3	1.5	100%	0	41.73	41.90	100%
EARS C3	2	100%	2	33.41	33.53	100%
EARS C3	2.5	100%	2	27.06	27.23	100%
EARS C3	3	100%	4	21.85	21.79	100%
EARS C3	3.5	100%	6	17.90	17.72	100%
EARS C3	4	97%	6	14.70	14.48	100%
EARS C3	5	97%	7	9.99	9.98	100%
EARS C3	6	88%	10	7.02	6.97	100%
EARS C3	8	66%	10	3.77	3.75	91%
CUSUM	2	100%	2	19.79	19.77	100%
CUSUM	3	94%	2	15.22	15.13	100%
CUSUM	4	94%	5	12.51	12.43	100%
CUSUM	5	94%	6	10.49	10.44	100%
CUSUM	6	94%	6	8.96	9.06	100%
CUSUM	8	94%	7	7.06	7.15	100%
CUSUM	10	94%	7	5.77	5.81	100%
CUSUM	12	94%	8	4.60	4.61	100%
CUSUM	14	94%	8	3.68	3.82	100%
CUSUM	17*	88%	9	2.97	2.91	100%
CUSUM	20	88%	10	2.38	2.34	100%
CUSUM	25	88%	12	1.75	1.79	100%

Delay = median days to detection; FA/DY = false alert events per district-year (4 diseases); NRS = share of districts detecting the national surge.

TABLE VI
BACKTEST (LAST 52 WEEKS): MAE, 80% COVERAGE, WIS RELATIVE TO SEASONAL NAIVE

Disease	h	MAE: naive	GLM	HGB	$\text{Cov}_{\text{naive}}$	Cov_{GLM}	rWIS_{GLM}	rWIS_{HGB}
dengue	1	52.3	36.6	33.8	82%	76%	0.52	0.52
dengue	4	55.4	57.7	43.5	81%	65%	0.80	0.55
diarrhoea	1	58.0	40.9	38.5	82%	83%	0.68	0.63
diarrhoea	4	59.6	46.9	42.8	81%	84%	0.76	0.71
ili	1	144.9	64.6	46.6	77%	75%	0.29	0.21
ili	4	151.2	221.6	166.4	77%	72%	1.03	0.91
malaria	1	3.4	3.0	3.4	88%	92%	0.68	0.83
malaria	4	3.6	3.8	4.0	87%	91%	0.90	0.95
all	1	64.7	36.3	30.6	82%	81%	0.41	0.35
all	4	67.5	82.5	64.2	82%	78%	0.94	0.80

C. Allocation vs. baseline and equity (AN-7, AN-8, C3)

With stock on 2025-12-31, closing every district's gap at the planning quantile would cost 0.5 M BDT; at a budget of 60% of that (0.3 M) the optimised plan with $\alpha = 0.10$ leaves weighted unmet need of 1,632 vs 6,727 patient-equivalents for the population-proportional plan, covers 57.4% vs 22.2% of the gap, and leaves 9 vs 22 districts below half coverage; unmet rapid-response team demand is 0 vs 0 teams. Table VII gives the efficiency–equity curve over six budgets. Rule: take the smallest α that leaves no district with a gap and nothing shipped, provided it costs at most 10% extra weighted unmet demand against the purely efficient plan; at 60% budget this gives $\alpha = 0.10$ (loss 9.8%, districts left with nothing 4→0). **Interpretation.** In this low-season snapshot (end of December) the shortfall is only ORS and IV saline (58% and 42% of its cost); per taka, ORS protects about four times more patient-equivalents than IV saline, so at 60% of the gap cost the plan spends the whole budget on ORS and ships no saline, which enters only from a budget of 80% of the gap, first where alerts or growth raise the weight. Population-proportional allocation spends the same money but ships saline and sachets to districts that already hold enough, which is why it meets less than half as much of the gap. Every extra step of α costs efficiency with diminishing equity gains (Table VII); a small floor ($\alpha = 0.10$) is enough to guarantee every district a share, and larger values mostly move supplies from districts with a gap to districts without one.

D. Scalability and Spark optimisations (ENG-2, ENG-3)

Table VIII reports the full `make all` (generator, load, analytics, export) at three scale factors on 4 vCPUs and 16 GB RAM (Spark local mode, 6 GB heap, Java 17): SF = 1 finishes in 19.6 minutes, well inside the 45-minute limit. Time grows roughly linearly with rows in the load stage, while analytics is almost constant because it works on weekly/daily aggregates whose size depends on districts and weeks, not on encounters. The Spark UI shows the silver stage as the bottleneck: it evaluates 16 rules, hashes every patient ID, shuffles once by `encounter_id` for resubmission handling and writes 48 partitions; the next step would be to fold the per-rule statistics into the write job instead of separate actions.

Two optimisations with before/after evidence (median of three runs on the SF = 1 lake, Spark UI screenshots in `docs/experiments/screenshots`): (i) **broadcast join of the facility registry** – with broadcasting disabled Spark plans a SortMergeJoin that shuffles every bronze encounter (Fig. 3: 28.9 M rows, 660 MiB of row data, written as 24 MiB of compressed shuffle files; the lake then also held the rehearsal drops) and sorts both sides; broadcasting the 783-row registry gives a BroadcastHashJoin with no shuffle of the large side: 6.68 s → 2.64 s (2.5×); (ii) **columnar storage** – the same per-facility, per-diagnosis count over all encounters reads 2.98 GB of CSV in the landing zone but only two of 17 columns from 0.33 GB of zstd Parquet in bronze: 9.44 s → 1.88 s (5.0×). (iii) **Partition pruning** on silver (epi_year×division) cuts the files scanned by a division-year query from 14 to 1, but the wall-time gain is small at this size (0.41 s → 0.37 s) because the table is already in the operating-system cache; it grows with data volume. Adaptive query execution (skew-join handling, coalescing of the 32 shuffle partitions) is enabled throughout; we did not run a separate skew experiment.

TABLE VII
EFFICIENCY–EQUITY CURVE (WEIGHTED UNMET NEED IN PATIENT-EQUIVALENTS; COVERAGE VALUED AT COST)

Budget	Plan	W. unmet	Gap cov. %	Min cov.	<50%
0.2	$\alpha = 0$	6,301	20.0	0%	22
0.2	$\alpha = 0.1$	6,475	19.1	0%	29
0.2	$\alpha = 0.25$	6,754	17.8	0%	28
0.2	$\alpha = 0.5$	7,244	15.4	0%	25
0.2	$\alpha = 0.75$	7,741	13.0	0%	31
0.2	$\alpha = 0.9$	8,045	11.5	0%	30
0.2	pop. baseline	9,036	7.7	2%	32
0.4	$\alpha = 0$	3,388	40.0	0%	12
0.4	$\alpha = 0.1$	3,563	38.3	0%	11
0.4	$\alpha = 0.25$	3,852	35.4	0%	10
0.4	$\alpha = 0.5$	4,346	30.5	0%	14
0.4	$\alpha = 0.75$	5,308	25.6	0%	21
0.4	$\alpha = 0.9$	6,024	22.5	0%	20
0.4	pop. baseline	7,809	15.0	5%	28
0.6	$\alpha = 0$	1,487	60.0	0%	9
0.6	$\alpha = 0.1$	1,632	57.4	0%	9
0.6	$\alpha = 0.25$	2,076	53.0	0%	9
0.6	$\alpha = 0.5$	2,823	45.6	0%	10
0.6	$\alpha = 0.75$	3,628	37.6	0%	11
0.6	$\alpha = 0.9$	4,353	32.2	0%	15
0.6	pop. baseline	6,727	22.2	7%	22
0.8	$\alpha = 0$	506	80.0	50%	0
0.8	$\alpha = 0.1$	692	75.7	9%	1
0.8	$\alpha = 0.25$	1,017	69.3	1%	4
0.8	$\alpha = 0.5$	1,496	59.8	1%	8
0.8	$\alpha = 0.75$	2,532	48.5	0%	10
0.8	$\alpha = 0.9$	3,341	40.5	0%	11
0.8	pop. baseline	5,791	28.9	9%	16
1	$\alpha = 0$	0	100.0	100%	0
1	$\alpha = 0.1$	150	94.1	50%	0
1	$\alpha = 0.25$	376	85.1	50%	0
1	$\alpha = 0.5$	903	71.6	7%	3
1	$\alpha = 0.75$	1,578	58.2	0%	9
1	$\alpha = 0.9$	2,625	47.5	0%	10
1	pop. baseline	4,980	34.8	11%	14
1.2	$\alpha = 0$	0	100.0	100%	0
1.2	$\alpha = 0.1$	0	100.0	100%	0
1.2	$\alpha = 0.25$	0	100.0	100%	0
1.2	$\alpha = 0.5$	423	83.3	50%	1
1.2	$\alpha = 0.75$	1,194	65.8	6%	4
1.2	$\alpha = 0.9$	1,962	54.1	-0%	9
1.2	pop. baseline	4,240	40.3	12%	13

TABLE VIII
SCALABILITY: SECONDS PER STAGE, TOTAL MINUTES, DATA SIZE (GB; THE SF = 1 LAKE ALSO HOLDS THE DELTA VERSIONS OF THREE LATER INCREMENTAL LOADS)

SF	Rows	Gen.	Bronze	Silver	Gold	Analyt.	Export	Total	Landing	Lake
0.25	7.2M	20	56	149	40	377	42	11.7	0.95	0.50
0.5	14.3M	31	70	230	45	401	44	14.1	1.70	0.92
1	28.6M	59	107	424	53	454	42	19.6	3.19	2.67

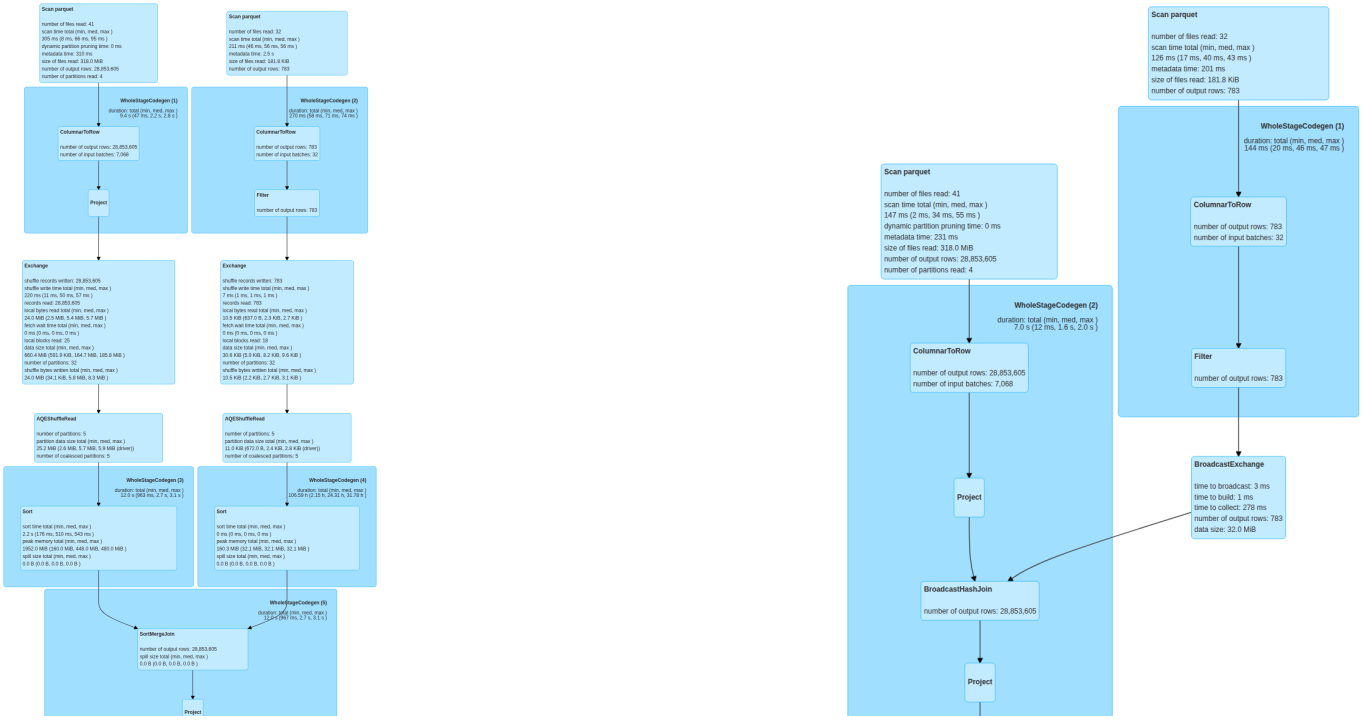


Fig. 3. Spark UI (History Server, SQL tab) for the registry join on the SF = 1 lake. Left: broadcast disabled – both sides are shuffled (Exchange) and sorted before a SortMergeJoin. Right: BroadcastHashJoin – the small facility registry is broadcast and the encounters are joined where they are read, with no shuffle of the large side.

E. Incremental loads, streaming and calendar

Incremental and idempotent (`ING-4`). Loading the first future daily drop into the SF = 1 lake added 13,420 encounters (including late records whose onsets fall in earlier epi-weeks) in 115 s, and the whole drop DAG including analytics and dashboard export took 10.2 minutes. Re-running the same day returned "nothing new" in 16 s; a forced re-ingest (112 s) left silver at 27,656,066 rows, all with distinct `encounter_id`, and every gold count unchanged. **Hidden-drop rehearsal.** Pointing `make drop` at the folder with all 14 generated days (same schema, new outbreaks, quality defects) skipped the day already loaded and ingested the other 13 days (177,864 rows, 2,823 quarantined) in 10.7 minutes end to end, regenerating alerts, forecasts, allocation and dashboard data without any code change. **Streaming (`ING-6`).** With the Structured Streaming job running (30-second trigger), a new encounter file dropped in the inbox refreshed `alerts.json` after a median of 87 s (maximum 129 s), inside the 5-minute requirement. These three measurements were taken during the earlier build, whose population and weather inputs were still synthetic stand-ins, and were not repeated on the final real-data lake. **Time travel (`ING-7`).** Every load creates a new Delta version of silver and gold, so "what did we know on day t" is a `versionAsOf` read (`make inspect WHAT=asof`). Reading `gold.weekly_counts` at the version before the rehearsal drop shows the counts known on 1 Jan 2026; the drop's late reports revised 15 of the 24 recent disease-weeks. The backtest does not keep a version per day: it rebuilds past knowledge from report dates (the reporting triangle). Rebuilding the same weeks from today's silver with *first report date* $\leq$ *1 Jan 2026* differs from the time-travel read by 54 of 162,427 cases, which validates that shortcut. Table IX shows the dengue calendar for the next 12 months.

TABLE IX
DENGUE PREPAREDNESS CALENDAR (NEXT 12 MONTHS)

Division	Campaign	Pre-position	Onset	Peak	End
Barishal	2026-04-13	2026-05-11	2026-06-08	2026-08-03	2026-11-30
Chattogram	2026-04-20	2026-05-18	2026-06-15	2026-09-14	2026-11-23
Dhaka	2026-04-13	2026-05-11	2026-06-08	2026-08-17	2026-11-30
Khulna	2026-04-20	2026-05-18	2026-06-15	2026-09-14	2026-11-23
Mymensingh	2026-03-30	2026-04-27	2026-05-25	2026-07-20	2026-11-23
Rajshahi	2026-03-23	2026-04-20	2026-05-18	2026-10-26	2026-11-23
Rangpur	2026-03-16	2026-04-13	2026-05-11	2026-09-07	2026-11-23
Sylhet	2026-03-30	2026-04-27	2026-05-25	2026-07-13	2026-11-23

VII. DASHBOARD DESIGN AND USABILITY

The dashboard (`VIS-1`) is plain HTML with D3.js v7 and topojson-client, vendored so it runs offline, and is started with one command (`run_dashboard.bat`, `./run_dashboard.sh` or `make serve`), which opens the browser already signed in. Fig. 4 shows the national view.

Coordinated views (`VIS-2`, `VIS-3`). (1) District choropleth of mean weekly incidence per 100,000 over the selected weeks with alert markers; (2) focus+context time series with the reported counts, the nowcast for weeks still being reported,

TABLE X
SEIR WHAT-IF SCENARIOS FOR THE RESPIRATORY SURGE

Scenario	Reported	Peak wk	Peak	Averted
no intervention (fitted)	373,348	8	68,388	0%
20% contact reduction at detection	306,443	10	38,264	18%
20% contact reduction 2 weeks earlier	306,443	10	38,264	18%
20% contact reduction 2 weeks later	308,014	9	39,289	17%
40% contact reduction at detection	133,051	17	8,946	64%
40% contact reduction 2 weeks earlier	133,051	17	8,946	64%
40% contact reduction 2 weeks later	168,533	10	11,575	55%

forecast median with 50/80/90% bands and the alert threshold (expected 7-day count + 2 SD); (3) seasonal heatmap, epi-week×year; (4) allocation view comparing forecast demand (10–90% interval and planning quantile), stock, recommended and baseline shipments; (5) ranked early-warning table; plus the what-if panel, the preparedness calendar, SEIR scenarios, the per-load data-quality report and an evaluation panel showing the detection trade-off and forecast skill. One shared state (disease, area, week range, resource, threshold, budget, α) is broadcast to every view with `d3.dispatch`: clicking a district on the map, a row in the table or a bar in the allocation view selects it everywhere; brushing the context strip or clicking a heatmap cell sets the period of the map and heatmap. Every view has tooltips, legends and axis labels.

Scale-aware design (`VIS-4`). The browser loads only the role's bundle of aggregated JSON: the national bundle is about 1.8 MB plus a 55 kB TopoJSON (limit 10 MB, checked by a test). Counts are at district×epi-week (the level at which district officers act and stocks are managed); forecasts, demand and allocations are per district for four weeks; detection is published as daily *statistics* for the last 180 days so the threshold can be changed in the browser without shipping counts.

What-if panel (`VIS-5`). Sliders for budget (20–120% of the gap), alert threshold and equity floor α look up a precomputed grid of optimiser runs (every combination solved offline, because the alert threshold changes the weights and team demand) and recompute alert status from the published statistics; the efficiency–equity curve highlights the current α.

Accessibility and honesty (`VIS-6`). Disease colours use a validated colour-blind-safe order (blue, orange, aqua, yellow; adjacent CVD $\Delta E \geq 9$ in both light and dark mode, checked with a palette validator), magnitudes use a one-hue blue ramp, alert states use reserved status colours always paired with an icon and a label; suppressed cells are hatched and shown as "<5"; every forecast carries its bands; every panel shows its data-freshness date and a banner states that the data are synthetic, the provenance of the public inputs and until when the weather is observed rather than climatology. **Mobile and playback (`VIS-7`).** The grid collapses to one column below 1,100 px and a "Play spread" button animates weekly incidence over the last year. **Roles (`PRV-3`).** A district officer's token returns a bundle with their own district in detail and all other areas as division aggregates. A "View as" menu switches role in the local demo mode; `-secure` turns off automatic sign-in.

Usability check. [The protocol in `docs/usability/usability_protocol.md` (six tasks, think-aloud, System Usability Scale) must be run with at least two non-CS users; insert their results and the changes made here.]

VIII. Ethical, Societal and Sustainability Considerations

Consequences of errors (A4). A missed outbreak costs lives; a false alarm costs money and, repeated, the trust that makes officials act on the next alert. We therefore report both error types, choose the operating point openly, route reporting-artefact alerts to "verify reporting" instead of "alert", and recommend that every alert is confirmed with the district before teams move. **Privacy.** The design follows minimisation and purpose limitation: identifiers stop at bronze, salted hashes cannot be reversed without a key that never enters the repository, exact ages are dropped, and the public-facing layer contains only suppressed aggregates; stigma is a real risk for small communities (e.g. malaria in hill districts), which is why small counts are hidden even at district level and why we recommend differential privacy for public releases. **Equity.** Allocating purely on expected unmet need concentrates supplies in Dhaka and the hill districts; the equity floor makes the trade-off explicit and gives policymakers a number to defend ("every district gets at least α of its population share"). Weights for alerts and growth are stated and adjustable; they must be agreed with the Ministry rather than set by engineers. **Misuse.** Alerts could be used to blame districts or facilities; the dashboard shows reporting context (back-fills, new facilities) to avoid penalising good reporting, and access is role-based and audited. **Automation bias.** Forecasts and plans are advisory: uncertainty bands, the baseline plan and the what-if panel invite human judgement; the model is interpretable by design (C5). **Sustainability.** The whole platform runs on one laptop-class machine in about 19.6 minutes per full rebuild and a few minutes per daily increment, uses open-source software only, stores data compressed (zstd Parquet) and recomputes only affected epi-weeks, keeping energy and cost low enough for district infrastructure; the same code scales out to a cluster or object store without changes. **Honesty about data.** Synthetic data can make methods look better than they will be on real reports (cf. the Google Flu lesson [6]); all results must be re-validated on Ministry data.

IX. Limitations and Future Work

(1) *Synthetic data.* Outbreak shapes, delays and artefacts follow our own generator, so detection and forecast accuracy are optimistic; validation on historical DGHS surveillance data (e.g. the 2019 and 2023 dengue seasons) is the first next step.

Weather after 24 Aug 2025 is station climatology (end of the NOAA mirror), and three of six age bands use an assumed split; the census table and Open-Meteo can replace both. (2) *Detection.* Weather-driven genuine rises (post-flood diarrhoea, early dengue seasons) count as false alerts against a ground truth that lists only injected outbreaks; a covariate-adjusted baseline (e.g. the forecast GLM's residuals) and Farrington-flexible [2] are natural extensions; slow outbreaks are partly absorbed by the adaptive level. (3) *Forecasting.* Direct models are refitted every four weeks, features at training time use complete data while prediction uses nowcasts, and intervals assume independent weeks. (4) *Optimisation* is single-period with linear costs; multi-period stock dynamics, lead times, cold-chain constraints and uncertainty (stochastic or robust programming) are future work. (5) *Privacy.* Complementary suppression protects the published hierarchy but not arbitrary external data; formal differential privacy is recommended for public releases. (6) *Engineering.* Local-mode Spark on one node; HDFS/Kubernetes deployment, upazila-level maps with stronger suppression, SMS alerts to district officers and integration with DHIS2 APIs are future work.

REFERENCES

[1] L. Hutwagner, W. Thompson, G. M. Seeman, T. Treadwell, "The bioterrorism preparedness and response Early Aberration Reporting System (EARS)," *J. Urban Health*, 80(1), 2003.

[2] C. P. Farrington, N. J. Andrews, A. D. Beale, M. A. Catchpole, "A statistical algorithm for the early detection of outbreaks of infectious disease," *J. R. Stat. Soc. A*, 159(3), 1996.

[3] D. F. Stroup, G. D. Williamson, J. L. Herndon, J. M. Karon, "Detection of aberrations in the occurrence of notifiable diseases surveillance data," *Stat. Med.*, 8(3), 1989.

[4] E. S. Page, "Continuous inspection schemes," *Biometrika*, 41(1/2), 1954.

[5] J. Bracher, E. L. Ray, T. Gneiting, N. G. Reich, "Evaluating epidemic forecasts in an interval format," *PLoS Comput. Biol.*, 17(2), 2021.

[6] D. Lazer, R. Kennedy, G. King, A. Vespignani, "The parable of Google Flu: traps in big data analysis," *Science*, 343(6176), 2014.

[7] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," *Commun. ACM*, 59(11), 2016.

[8] M. Armbrust et al., "Delta Lake: High-performance ACID table storage over cloud object stores," *PVLDB*, 13(12), 2020.

[9] M. Bostock, V. Ogievetsky, J. Heer, "D3: Data-Driven Documents," *IEEE TVCG*, 17(12), 2011.

[10] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness Knowl.-Based Syst.*, 10(5), 2002.

[11] World Health Organization, *Technical Guidelines for Integrated Disease Surveillance and Response (IDSR)*, 3rd ed., 2019.

[12] R. B. Cleveland, W. S. Cleveland, J. E. McRae, I. Terpenning, "STL: A seasonal-trend decomposition procedure based on Loess," *J. Official Stat.*, 6(1), 1990.

[13] Q. Huangfu, J. A. J. Hall, "Parallelizing the dual revised simplex method," *Math. Prog. Comput.*, 10, 2018.

[14] C. Dwork, F. McSherry, K. Nissim, A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. TCC*, 2006.

[15] W. O. Kermack, A. G. McKendrick, "A contribution to the mathematical theory of epidemics," *Proc. R. Soc. A*, 115(772), 1927.

[16] T. White, *Hadoop: The Definitive Guide*, 4th ed. O'Reilly, 2015.

[17] J. Leskovec, A. Rajaraman, J. D. Ullman, *Mining of Massive Datasets*, 3rd ed. Cambridge Univ. Press, 2020.

[18] T. Erl, W. Khattak, P. Buhler, *Big Data Fundamentals*. Prentice Hall, 2016.

[19] O. J. Brady et al., "OpenDengue: data from the OpenDengue database," V1.3, 2024. [Online]. Available: https://opendengue.org

[20] UN OCHA, "Bangladesh – Subnational Administrative Boundaries (COD-AB)," HDX, v03, 2023.

[21] NOAA National Centers for Environmental Information, "Global Surface Summary of the Day (GSOD)," NOAA Open Data Dissemination (AWS `noaa-gsod-pds`), accessed 2026.

[22] Facebook Connectivity Lab and CIESIN, "High Resolution Settlement Layer (HRSL)," Data for Good at Meta (AWS `dataforgood-fb-data`), CC BY 4.0, accessed 2026.

[23] P. Zippenfenig, "Open-Meteo.com Weather API," Zenodo, 2023, doi:10.5281/zenodo.7970649.

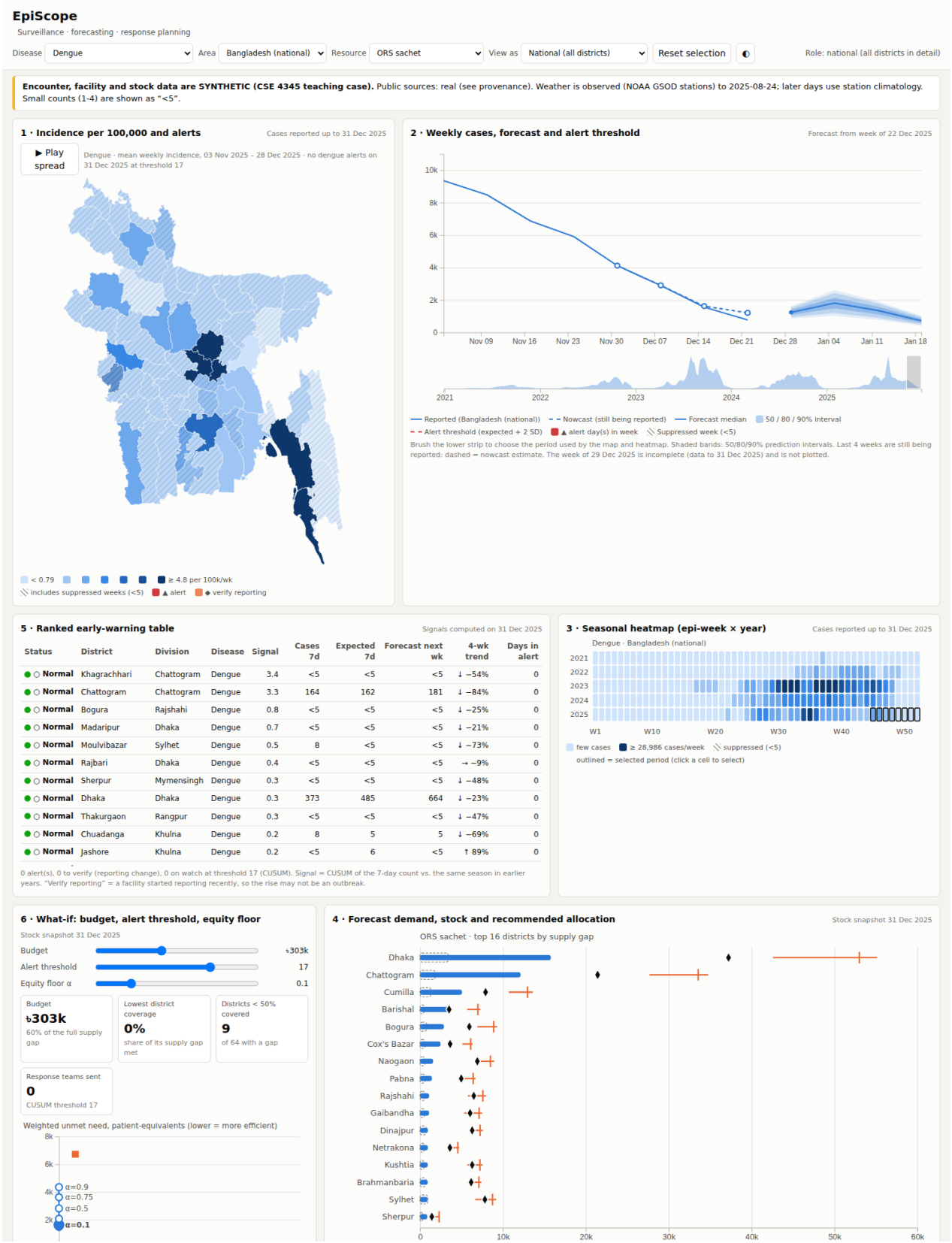


Fig. 4. EpiScope dashboard, national role (SF = 1 build). Views are linked: selecting a district on the map updates the time series, heatmap, table and allocation view.

APPENDIX A
POLICY BRIEF FOR THE DIRECTOR, DISEASE CONTROL & PREPAREDNESS UNIT

Seeing outbreaks sooner and sending supplies where they save most lives

EpiScope prototype – summary for decision-makers

Important: this prototype was built and tested on realistic *simulated* patient records for all 64 districts (2021–2025), because real patient data cannot be used for a student project. The findings below show what the system can do; the numbers must be re-checked on the Ministry's own data before any operational decision.

1. The current situation

Information about dengue, diarrhoea and cholera, influenza-like illness and malaria reaches the Ministry late, in many formats, and is mostly analysed after the fact. As a result, saline, test kits and teams often reach a district after its hospitals are already full. EpiScope brings the routine reports from hospitals and health complexes, the weather record, population figures and district stock levels together every day and turns them into four things officials can act on: **early warnings**, **forecasts for the next four weeks**, **a recommended distribution of supplies and response teams**, and **a calendar for the year ahead**.

2. Top three recommendations

1) **Adopt a weekly "warning review" based on the ranked warning list.** In our test the warning system recognised 88% of the simulated local outbreaks, typically about 9 days after they began, while producing roughly 2.97 unnecessary warnings per district per year across the four diseases. It also ignored a hospital that suddenly sent three weeks of late reports at once – a situation that fools simpler systems. *Expected impact:* response teams can be sent days to weeks earlier than with monthly spreadsheet reviews, with a manageable number of false alarms. Each warning should be confirmed by a phone call to the district before a team is sent.
2) **Distribute supplies with the recommended plan, guaranteeing every district a small minimum share** (at least one tenth of its share by population of every shipment). With a budget of 60% of what would be needed to fill every district's shortfall, the recommended plan fills 57.4% of the districts' combined shortfall, against 22.2% if supplies are shared by population alone, and guarantees every district at least a share of each shipment. This guarantee costs a little efficiency (about 9.8% more unmet need than a plan that ignores fairness) but means no district is left with nothing. *Expected impact:* more saline and test kits where patients will actually be, without abandoning low-risk districts.
3) **Follow the preparedness calendar: start dengue prevention between mid-March and late April, and have supplies in place by May.** In every division dengue cases begin to rise in May or June and fall away by late November; the peak comes first in Sylhet and Mymensingh (July) and last in Rajshahi (late October). Diarrhoea rises from March and peaks between April and June. The calendar tells each division when to begin larval control (about eight weeks before the rise), when supplies must be in place (four weeks before) and when to add hospital staff (two weeks before the peak). *Expected impact:* the heaviest weeks of the season are met with stock already in place.

3. How to read the dashboard

- **Map:** darker blue = more cases per 100,000 people in the chosen weeks. A red triangle marks a district with an active warning; an orange diamond means "check the reporting first" (for example, a new hospital has just joined the system). Striped areas include small numbers that are hidden for privacy.
- **Weekly chart:** the solid line is what has been reported; the dashed end is our estimate for the weeks still being reported; the shaded fan is the forecast – the darkest band is where the true number is most likely to fall, the lightest covers 9 out of 10 outcomes. The red dotted line is the warning threshold.
- **Warning list:** districts at the top need attention first. "Days in alert" shows how long the warning has lasted.
- **Supplies chart and sliders:** compare expected need (orange), current stock (black diamond) and the recommended shipment (blue). The sliders let you ask "what if the budget were smaller?" or "what if we guarantee every district a larger minimum share?" and see the effect immediately.
- Every panel shows the date of its latest data. A district officer sees their own district in full and the rest of the country only in summary.

4. Confidence and limitations

- Forecasts one week ahead were much closer to what happened than simply assuming "same as this week last year": their error score was only 0.41 times that of the simple method, and the stated ranges contained the real number about as often as promised (8 times in 10). Forecasts become less certain further ahead, and the fans in the chart show this honestly. Four-week forecasts should guide preparation, not exact orders.

- Slow-building outbreaks are harder to spot than sudden ones; unusual weather (floods, an early monsoon) also causes genuine rises that the system will flag and that officials must interpret.
- The results depend on hospitals reporting regularly. The system detects facilities that stop reporting, but it cannot see patients who never reach a facility.
- Supply needs are converted from cases using standard assumptions (for example, six bags of saline per admitted dengue patient); these should be confirmed with clinicians and stores staff.
- Some public inputs (weather, population) must be refreshed from the official sources when the system is installed at the Ministry.

5. Data-protection assurance

- Names and national ID numbers never leave the secure first storage area: they are replaced by an irreversible code made with a secret key that is kept outside the software.
- Exact ages are reduced to age groups. The dashboard and every export show only totals per district and week; any count between 1 and 4 is shown as “fewer than 5”, and a second number is hidden when totals would otherwise reveal it.
- Access is by personal key, each user sees only what their role requires, and every access is logged.
- Any record-level file shared with researchers is checked so that each combination of place, time, age group and sex covers at least five people; for public releases we recommend adding a small amount of statistical noise (differential privacy).

Appendix B
Contribution Matrix

To be completed by the group: enter L (lead), S (support) or – (no involvement) for each member and area, agree the overall percentage shares (total 100), copy the commit counts from `git shortlog -sn` and sign. The individual moderation factor is $\pi_i = \text{clip}(3\sigma_i/100, 0.8, 1.1)$. The matrix must be consistent with the Git history and with `AI_USAGE.md`.

Work area	**Partha Nath**	**Tahmid Jamal**	**Taofiq Omar Raju**
Problem analysis, proposal, conflict analysis	Lead		
Synthetic generator and public-data acquisition	Lead		
Ingestion, lakehouse layers, incremental loads	Lead		
Data quality, lineage and privacy controls	Lead		Serving privacy
Outbreak detection and evaluation		Lead	
Forecasting, backtesting and demand conversion		Lead	
Optimisation model and preparedness calendar		Lead	
Serving layer and D3.js dashboard			Lead
Scalability experiment and Spark optimisations	Lead		
Report and policy brief	Integration, architecture	Analytics sections	Dashboard sections
Video production	Edit; chapters 1–3	Chapter 4	Chapters 5–6
Agreed overall share σ_i (%, total 100)			
Commits / lines changed (from `git shortlog -sn`)	26 commits	1 commit	2 commits
Signature			

APPENDIX C
INDIVIDUAL REFLECTIONS

Partha Nath (0222220005101104) – Data Engineer

I led the data platform: the synthetic generator, the bronze, silver and gold layers on Delta Lake, the 27 data-quality rules with quarantine, pseudonymisation, and the DAG that runs everything with one command. I also integrated the report and produced the video. The hardest problem was making every load idempotent. A file can arrive twice, a record can be resubmitted, and a late report can change an old week. A SHA-256 manifest in bronze and a Delta MERGE on the encounter ID in silver solve this, and the recorded run shows it: loading the same day again returned "nothing new", and at full scale the number of silver rows equalled the number of distinct encounter IDs (27,656,066). One decision I would revisit is the partitioning. Year and division keep the folder count small (48 instead of 384), but every daily drop touches all eight divisions of the current year, so I would measure a layout by load date with periodic compaction before the next version. Recording the run also taught me to check the environment first: my first attempt failed because Docker Desktop was not running, and the scale factor has to be set again in every new terminal. Writing each conflict as a number, for example at most three false alerts per district and year, gave the team a concrete target to test against instead of a matter of opinion. We used Claude through Claude Code for first-draft code and text, as AI_USAGE.md records; I ran the pipeline myself and am responsible for explaining every part of my area.

Tahmid Jamal (0222220005101083) – Analytics Engineer

I led the analytics: outbreak detection, forecasting, demand conversion, the allocation optimiser, the preparedness calendar and the SEIR scenarios, all reading only the gold tables. The hardest problem was choosing a detector honestly. Early designs raised far too many false alerts, and the planted reporting artefacts, a facility that goes silent and then back-fills weeks of reports, and a facility that starts reporting mid-way, produced alerts a district officer would chase for nothing. Guarding the signal against those artefacts and applying a fixed rule, the highest sensitivity with at most three false alerts per district-year, gave CUSUM at threshold 17: 87.5% of local outbreaks, a median delay of nine days and 2.97 false alerts, against 34–66% for the EARS baselines at a similar false-alarm level. The evaluation uses the injected ground truth, which the detector never reads, so the numbers are not tuned to the answer. One thing I would change is the tie-break by shorter delay: it was added after seeing the results, which we state openly, and next time I would fix it in advance. The forecasts also show their limits. One week ahead the GLM nearly halves the seasonal-naive error (36.3 against 64.7 cases), but for influenza-like illness four weeks ahead it does worse than last year, so a hierarchical model is the obvious next step. For non-experts, the equity floor was easiest to explain as one trade-off: $\alpha = 0.1$ costs 9.8% more unmet need but leaves no district with nothing. Claude drafted much of the first code, as AI_USAGE.md records; I am responsible for understanding and explaining every result in Chapter 4 of the video and in the results section of this report.

Taofiq Omar Raju (0222220005101100) – Visualisation Engineer

I led the serving layer and the D3.js dashboard: the linked views, the what-if panel, role-based bundles, small-cell suppression and the audit log. The hardest problem was privacy in every output, not only on the map. Hiding counts from one to four is easy; the difficulty is that a hidden cell can be recovered by subtracting the visible cells from a district, division or national total. Complementary suppression hides a second cell in those cases, and automated tests scan every export for counts from one to four and re-check the published totals, which is how we know the rule holds. Speed was the second problem. Instead of solving the optimiser in the browser, all 288 slider combinations are solved beforehand, so the what-if panel responds instantly. One decision I would change is the "View as" switch: it is useful for marking and the video, but it exists only in demo mode, and a real deployment should give each officer their own token. Some behaviour also looks like a bug to a first-time user, such as a heatmap click narrowing the time series to a single week, and it needs a clearer cue. The usability test with non-CS users is still to be run, and its results will decide what we change first. Designing for district officers taught me that every status needs an icon and a label, not only a colour, and that every panel must say how fresh its data are. Claude produced first-draft code, as AI_USAGE.md records; I am responsible for understanding and explaining every part of the serving layer and the dashboard.